

Анализ кода в ДЗ

Иванова Татьяна Владимировна
tvivanova@itmo.ru

Все на одной строке, отсутствие скобок – плохой стиль!

```
if (value == 0) throw std::invalid_argument("Division by zero.");
```

```
for (auto& elem : m_data) elem /= value;
```

```
int Sample<PAR>::GetSizeX() const { return m_size_x; }
```



```
if (value == 0)  
    throw std::invalid_argument("Division by zero.");
```

```
for (auto& elem : m_data) {  
    elem /= value;  
}
```

```
int Sample<PAR>::GetSizeX() const  
{  
    return m_size_x;  
}
```

- Плохо для дебага
- Плохо для понимания кода другим программистом
- Плохо для проверки преподавателем!!!
- Потенциальный источник проблем при изменении кода

Условие / исключение / assert

- Вариант 1:
 - простое условие
 - используется, если понятно как реагировать (сообщение на экран, MessageBox, и т.д.)
 - часто используется при проверке входных данных от пользователя
- Вариант 2:
 - условие и создание исключения
 - самый универсальный (работает не только в отладке, можно перехватить в вызывающей функции, и т.д.)
 - медленнее
- Вариант 3:
 - если условие НЕ выполняется – в Debug возникает сообщение об ошибке
 - не влияет на скорость в Release
 - используется **только** в режиме отладки программы (ошибки быть не должно, но вдруг)

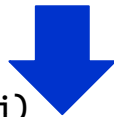
```
#include <cassert>
void Function(double* array)
{
    // вариант 1 -----
    if (array == 0)
    {
        cout << "error" << endl;
    }
}
```

Try – catch

```

PAR& Sample<PAR>::operator()(int i, int j)
{
    try
    {
        if ((i >= m_size_x) || (j >= m_size_y) || (i < 0) || (j < 0))
            throw std::string("index is out of bounds.");
    }
    catch (const std::string& err)
    {
        std::cout << err << '\n';
        if ((i >= m_size_x) && (j < m_size_y))
            return m_data[m_size_x * j + m_size_x - 1];
        ...
    }
    return m_data[m_size_x * j + i];
}

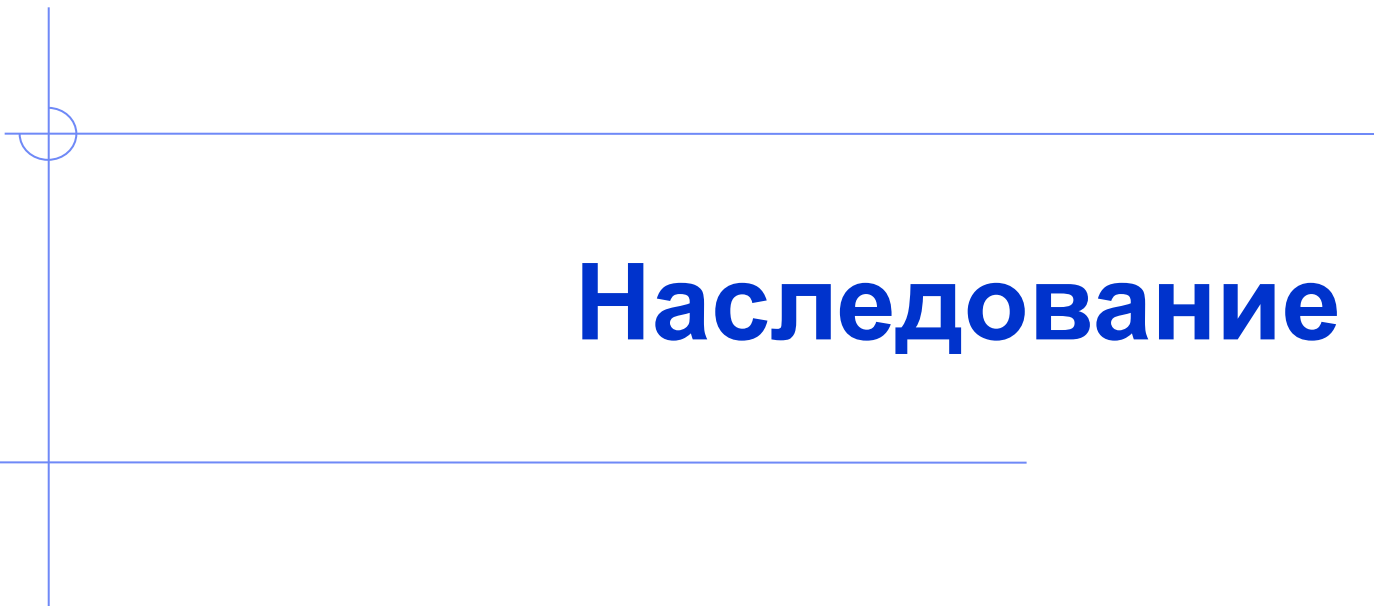
```



```

PAR& Sample<PAR>::operator()(int i, int j)
{
    if ((i >= m_size_x) || (j >= m_size_y) || (i < 0) || (j < 0))
        throw std::string("index is out of bounds.");
    return m_data[m_size_x * j + i];
}

```



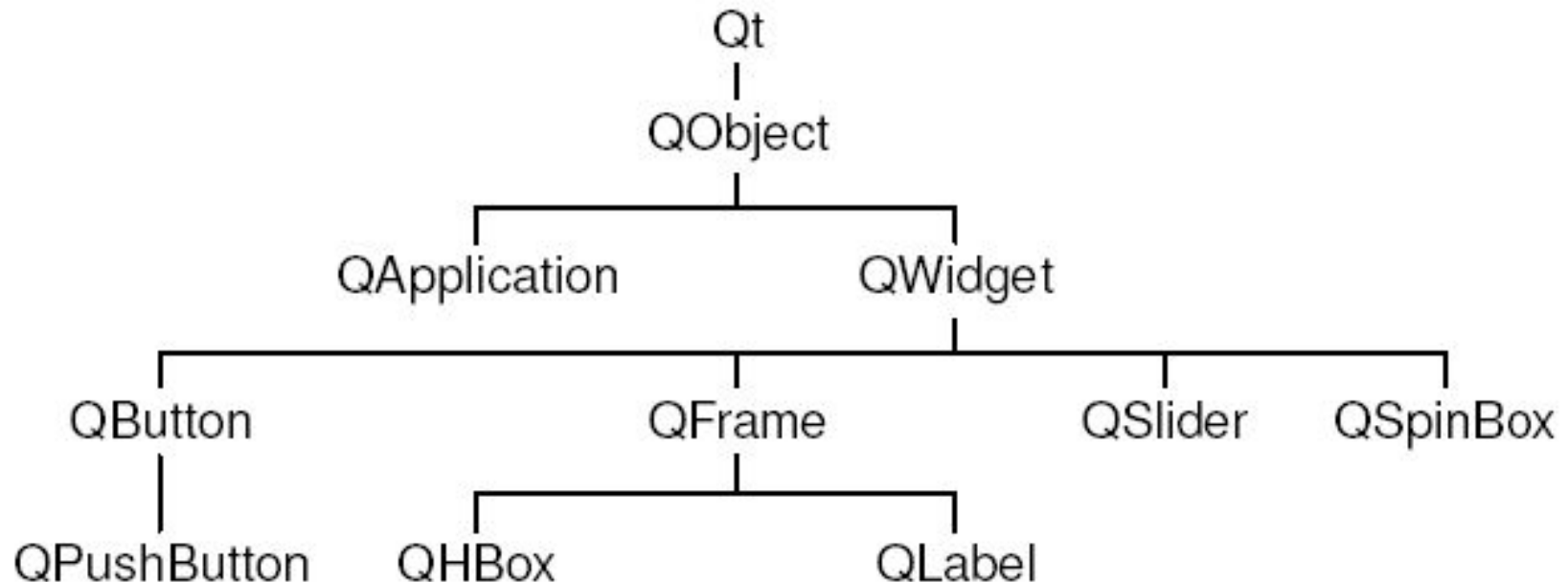
Наследование



Иванова Татьяна Владимировна
tvivanova@itmo.ru

Наследование

- **Наследование** – такое отношение между классами когда один из них наследует (повторяет) структуру и поведение другого
 - содержание (переменные-члены)
 - поведение (функции-члены)



Пример наследования: линза и зеркало

```
class Lens
{
    Position m_p;
    Media m_media;
    double m_R1, m_R2;
    double m_d, m_D;
};
```

```
class Mirror
{
    Position m_p;
    double m_R;
    double m_D;
};
```

Пример наследования: деталь

// базовый класс

```
class Detail
{
protected:
    Position m_p;
    double m_D;
};
```

// класс-наследник

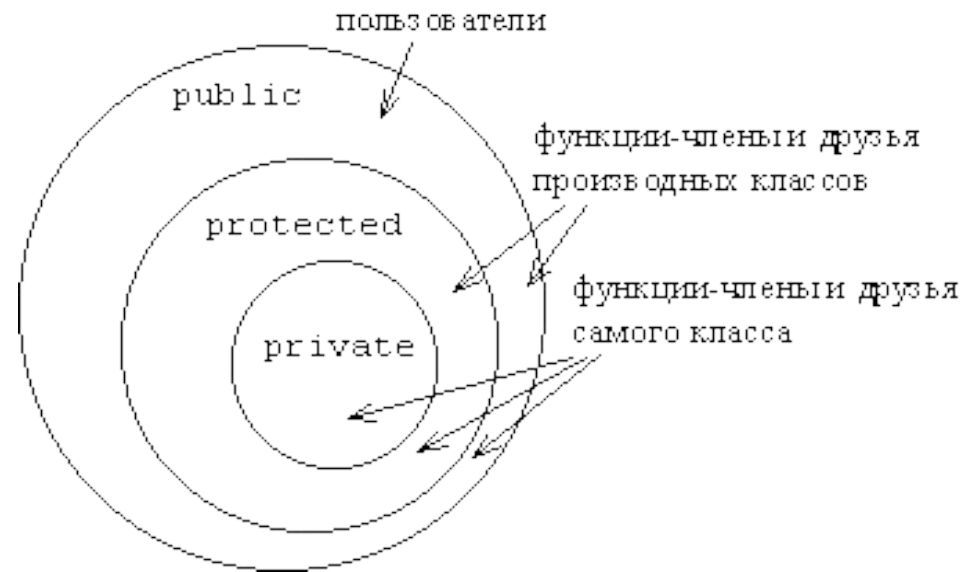
```
class Lens : public Detail
{
private:
    Media m_media;
    double m_R1, m_R2;
    double m_d;
};
```

// класс-наследник

```
class Mirror : public Detail
{
private:
    double m_R;
};
```


Доступ к членам класса

	public (открытые)	protected (защищенные)	private (закрытые)
функции-члены и друзья класса	+	+	+
функции-члены и друзья производных классов	+	+	—
пользователи	+	—	—



Типы наследования

- `class X : public Y;`
 - наследник является подтипом и выполняет все обязательства родителя
 - допустимо преобразование X^* в Y^*
- `class X : private Y;`
 - наследник не будет подтипом родителя
 - `public` и `protected` члены родителя станут `private` членами наследника
 - преобразование X^* в Y^* допустимо для друзей и членов `X`
 - последующее наследование не имеет смысла
- `class X : protected Y;`
 - наследник не будет подтипом родителя
 - `public` и `protected` члены родителя станут `protected` членами наследника
 - преобразование X^* в Y^* допустимо для членов и друзей и наследников `X`

Доступ к переменным-членам базового класса

	public (открытые)	protected (защищенные)	private (закрытые)
public-наследование	public	protected	недоступны
protected-наследование	protected	protected	недоступны
private-наследование	private	private	недоступны

Последовательность вызовов конструкторов и деструкторов

- При создании экземпляра класса Lens
 - сначала вызовется конструктор базового класса Detail()
 - потом конструктор класса Lens()
- При разрушении экземпляра класса Lens деструкторы будут вызваны в обратном порядке:
 - сначала деструктор наследника ~Lens()
 - потом деструктор базового класса ~Detail()
- В конструкторе наследника с параметрами следует вызывать конструктор базового класса
 - (по умолчанию будет вызван конструктор базового класса без параметров)

Множественное наследование

```
class Base1
{
public:
    Base1();
    ~Base1();
};
```

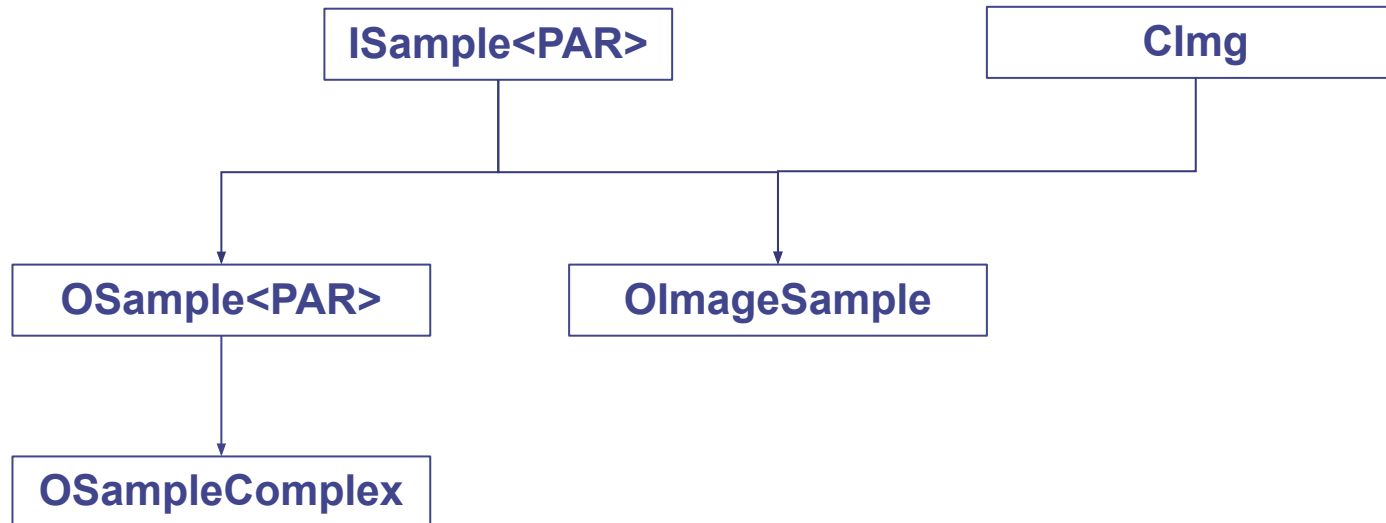
```
class Base2
{
public:
    Base2();
    ~Base2();
};
```

```
class Child : public Base1, public Base2
{
public:
    Child();
    ~Child();
};
```

- при создании экземпляра класса Child: Base1(), Base2(), Child()
- при разрушении экземпляра класса Child: ~Child(), ~Base2(), ~Base1()

Пример множественного наследования

- См. otf_lomo



Виртуальные функции

- **Виртуальные функции** – функции базового класса, которые можно заместить в каждом производном классе
 - позволяют избавиться от переменных-членов, представляющих собой флаг или тип отвечающий за внутреннее строение класса
 - всегда будет вызываться функция наследника

Виртуальная функция в базовом классе

```
class Detail
{
public:
    virtual void print() const;
};
////////////////////////////////////
void main ()
{
    // виртуальные функции важны, когда создаются указатели
    на класс
    Detail *p=new Lens;
}
```


Перегруженные функции в классах-наследниках

```
Detail *l=new Lens;
```

- Если функция-член базового класса должна подменяться функцией-членом класса-наследника, то при ее объявлении используется модификатор `virtual`

```
l.print(); // Lens::print
```

- Если должны вызываться последовательно обе функции, то это необходимо сделать принудительно вызвав функцию-член базового класса

```
Detail::print();
```

Абстрактные классы

- Абстрактный класс – класс, имеющий чисто виртуальные функции
 - чисто виртуальная функция – функция не определена в базовом классе, и обязательно должна быть перегружена в наследниках
 - экземпляр абстрактного класса создать нельзя, но можно создать экземпляры его наследников

```
// чисто виртуальный класс Деталь
class Detail
{
protected:
    double m_z, m_D;

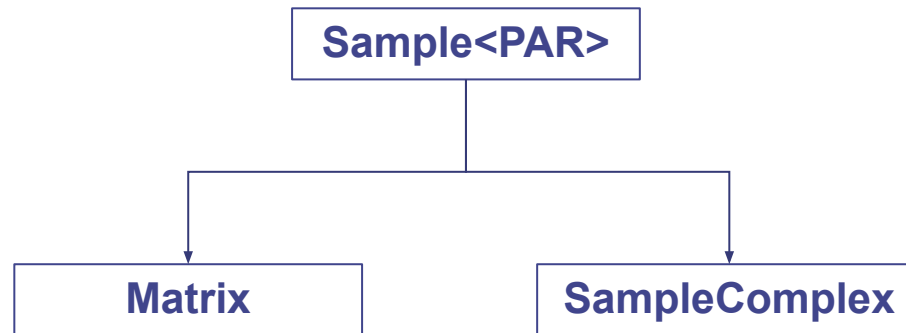
public:
    // ...

    // вычисление хода луча
    virtual void RayTrace() = 0;
};
```

Лабораторная работа №4

(часть 1)

- Часть 1 - Реализовать классы Matrix и SampleComplex



Класс SampleComplex

```
class SampleComplex: public Sample<std::complex<double>>
```

- полный конструктор для квадратной выборки
- вычисление:
 - модуль
 - фаза
 - интенсивность (квадрат модуля)
 - (записывается в вещественную часть, мнимая часть обнуляется)
- преобразование из SampleComplex в Sample<double> и из Sample<double> в SampleComplex
- функции вывода в файл:
 - только вещественной части выборки
 - только мнимой части выборки
 - модуля выборки
 - фазы выборки
 - интенсивности выборки (квадрат модуля)

Класс Matrix

```
class Matrix: public Sample<double>
```

- создание единичной матрицы
- оператор * (матричное произведение)
- оператор ~ (транспонирование)
- операторы /=, *= - генерация исключения с сообщением об ошибке
- проверка возможности нахождения произведения
 - те функции, которые были реализованы в базовом классе не забываем делать виртуальными

Библиотеки

Иванова Татьяна Владимировна
tvivanova@itmo.ru

Библиотеки

- Подключение в виде исходников (обычно шаблоны)
 - boost (www.boost.org) - файловая система, регулярные выражения, линейная алгебра, текст, и т. д.
 - QCustomPlot (<https://www.qcustomplot.com/>) – графическая библиотека на основе Qt для визуализации данных (графики, диаграммы, карты уровней)
- Подключение в виде готовой dll
 - FFTW (<https://www.fftw.org/>) – вычисление преобразования Фурье
 - Qt (<https://www.qt.io/>) – библиотека для создания пользовательского интерфейса (нужен VPN)
- Сборка dll из исходников, и потом подключение dll
 - Ceres Solver (<http://ceres-solver.org/>) – оптимизация
 - OpenCV (<https://opencv.org/>) - компьютерное зрение, машинное обучение и обработка изображений
 - Qt (если нет VPN)
 - Qwt (<https://qwt.sourceforge.io/>) – графическая библиотека на основе Qt для визуализации данных (графики, диаграммы, карты уровней)

Параметры

- Тип лицензии
 - Бесплатные
 - Бесплатные для некоммерческого использования
 - Платные
- Как подключать (dll / исходники)
- Объем
- Сложность в использовании
- Надежный разработчик
- Что принято в группе программистов (компании где вы работаете)
- Когда нужны библиотеки
 - с одной стороны – не делаем сами то, что можно взять из библиотеки
 - с другой стороны – не подключаем сложную и объемную библиотеку для простых вещей

При использовании библиотек

- При подключении dll **важно** проверить соответствие своему проекту
 - Платформа
 - Компилятор (пример – Qt)
 - 32x / 64x
 - Debug / Release
- Следим за версиями, если работаем в группе – обновления версий синхронизируем
 - хотя бы на уровне используемых функций и их параметров



Подключение в Visual Studio библиотеки FFTW

- В переменная окружения PATH
 - добавить путь к библиотеке, например:
`C:/lib/fftw64`
 - http://aco.ifmo.ru/el_books/programming/library/fftw.html
- меню Project->[имя проекта]Properties:
 - Выбрать конфигурацию "All Configurations"
 - В разделе "Свойства конфигурации -> Каталоги VC++" добавить в "Каталоги включения" и "Каталоги библиотек" путь, например:
`C:/lib/fftw64;`
 - В разделе "Свойства конфигурации -> Компоновщик -> Ввод" добавить в "Дополнительные зависимости" имя библиотеки lib
`libfftw3-3.lib;`
- Для boost – **только** "Каталоги включения"

Подключение через CMake

- На следующей лекции

Лабораторная работа 4 (часть 2)

- Подключить библиотеки boost и FFTW ([см.сайт](#))
- Сравнить перемножение матриц при помощи своего проекта и при помощи boost::numeric::ublas
- Выполнить преобразование Фурье от двумерной комплексной выборки SampleComplex 512x512, заполненной единичными значениями, вывести результат в файл
 - пример [см.сайт](#)
 - использовать функцию GetPointer(); из Sample
 - `(fftw_complex*)data.GetPointer()`
 - результат – дельта-функция в элементе (0,0)

[illegible]